Transformer

W ostatnim notebooku poznaliśmy mechanizm Attention, który pozwala transformerom skupić się na zależnościach słów. Dzisiaj poznamy całą architekturę i przebieg danych w transformerze.

Przypomnienie: Co już wiemy

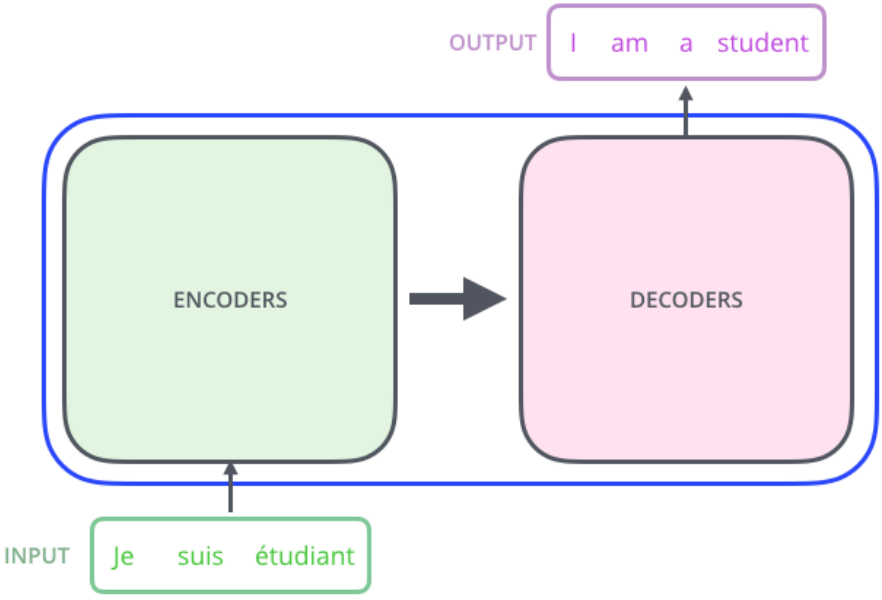
- Attention pozwala każdemu tokenowi „spojrzeć" na inne tokeny w sekwencji
- Multi-Head Attention umożliwia jednoczesne skupienie się na różnych aspektach relacji między tokenami



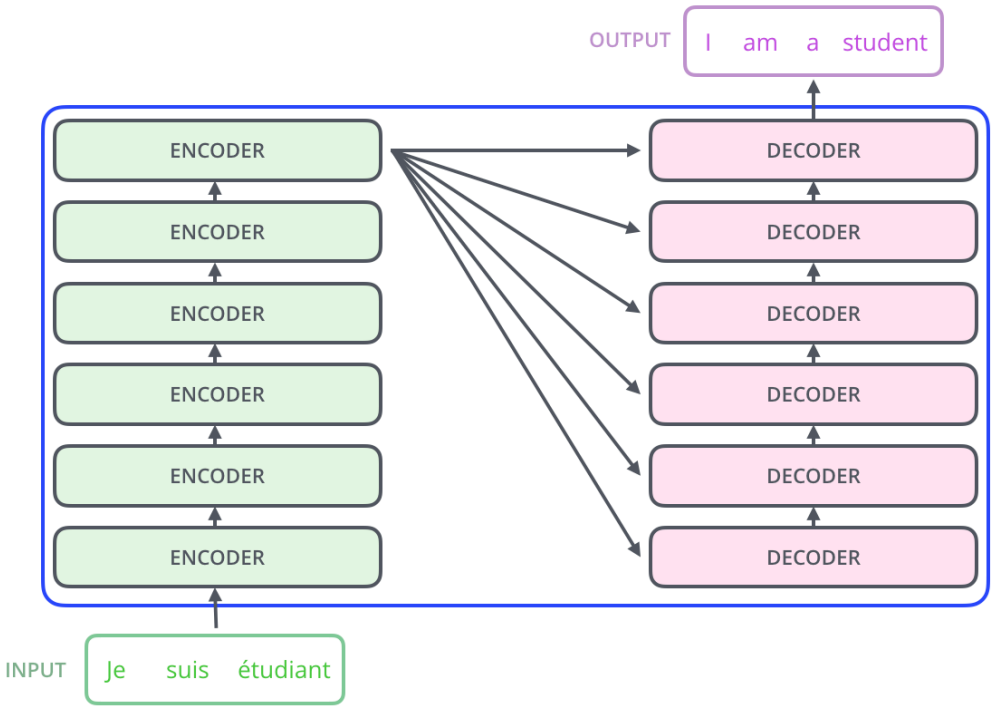
Architektura Transformera

Pełny model składa się z dwóch części:

1. **Encoder** – przetwarza wejściową sekwencję (np. zdanie źródłowe).
2. **Decoder** – generuje wyjściową sekwencję (np. tłumaczenie).

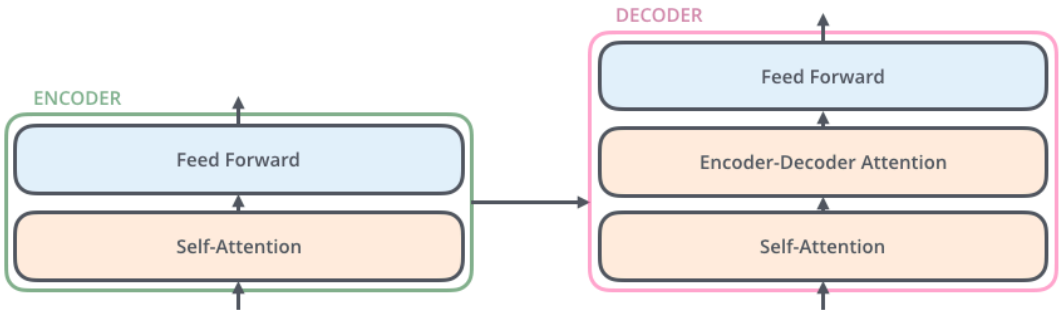


Enkoderów może być dowolna ilość. Po drugiej stronie mamy dekodery - tyle samo co enkoderów.



Każdy z nich zawiera powtarzające się bloki (warstwy):

- Encoder Layer = Multi-Head Attention + Feed Forward
- Decoder Layer = Masked Multi-Head Attention + Encoder-Decoder Attention + Feed Forward



Zanim jednak wejdziemy do pierwszego encodera musimy zembedingować nasz input - i co ważne pamiętać o kolejności

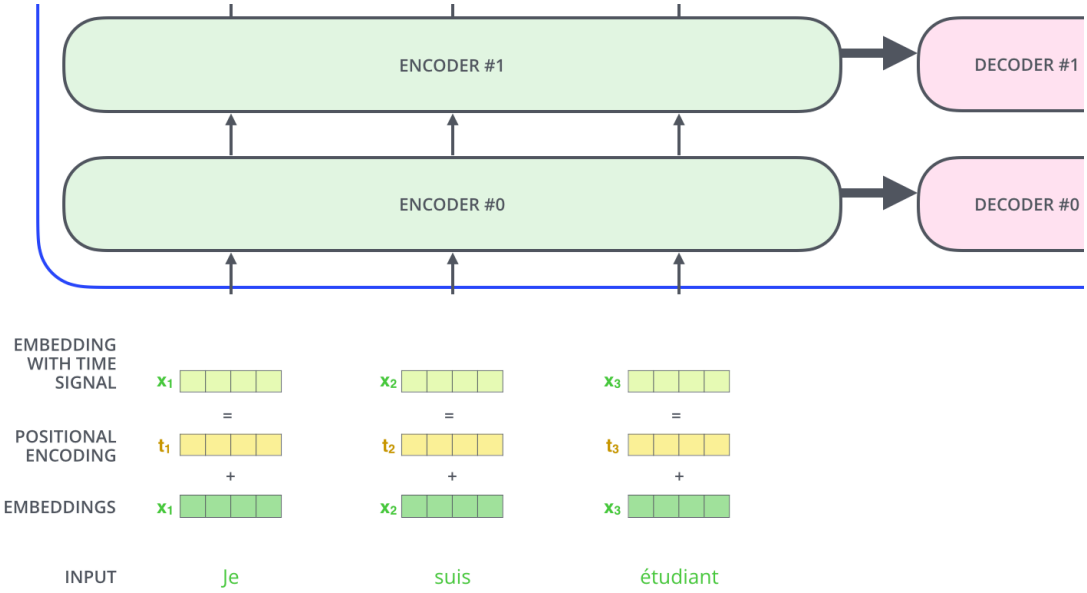
Flow Transformera

In [2]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
```

1 Positional Encoding

Transformery, w przeciwieństwie do modeli sekwencyjnych (RNN, LSTM), **nie przetwarzają tokenów w kolejności** — widzą całą sekwencję naraz.
Dlatego same z siebie **nie wiedzą**, które słowo było pierwsze, a które drugie.

Aby temu zaradzić, do każdego wektora wejściowego (embeddingu słowa) dodajemy **wektor pozycyjny** (*positional encoding*), który zawiera informację o miejscu tokena w zdaniu.



Model dodaje do wektora embeddingu słowa `E_i` wektor pozycyjny `P_i`:

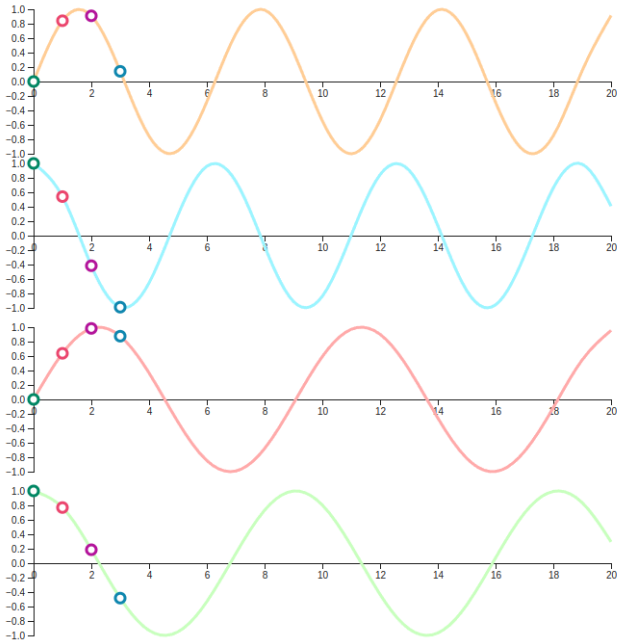
X_i = E_i + P_i

Każdy `P_i` jest deterministycznie obliczany (nie uczymy modelu tego)

Jak to działa?

1. Każda pozycja w zdaniu (np. 0, 1, 2, 3, ...) ma swój unikalny wzorzec liczb, który tworzymy za pomocą fal sinus i cosinus o różnych częstotliwościach.
2. Bierzemy numer pozycji tokena w zdaniu (np position = 0)
3. Dla każdego wymiaru embeddingu (np. 64) liczymy dwie wartości: sin i cos.
   czyli:

- pos – numer tokena w sekwencji,
- i – numer wymiaru w wektorze (np. 0...63),
- d_model – całkowity rozmiar wektora (np. 64)



	p0	p1	p2	p3	
	0.000	0.841	0.909	0.141	i=0
	1.000	0.540	-0.416	-0.990	i=1
	0.000	0.638	0.983	0.875	i=2
	1.000	0.770	0.186	-0.484	i=3

Positional Encoding

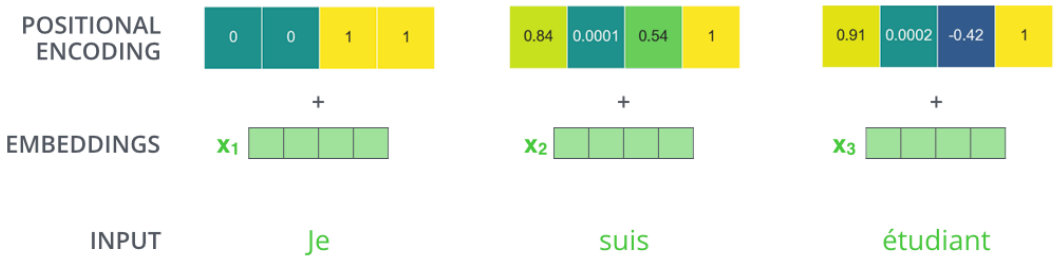
$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

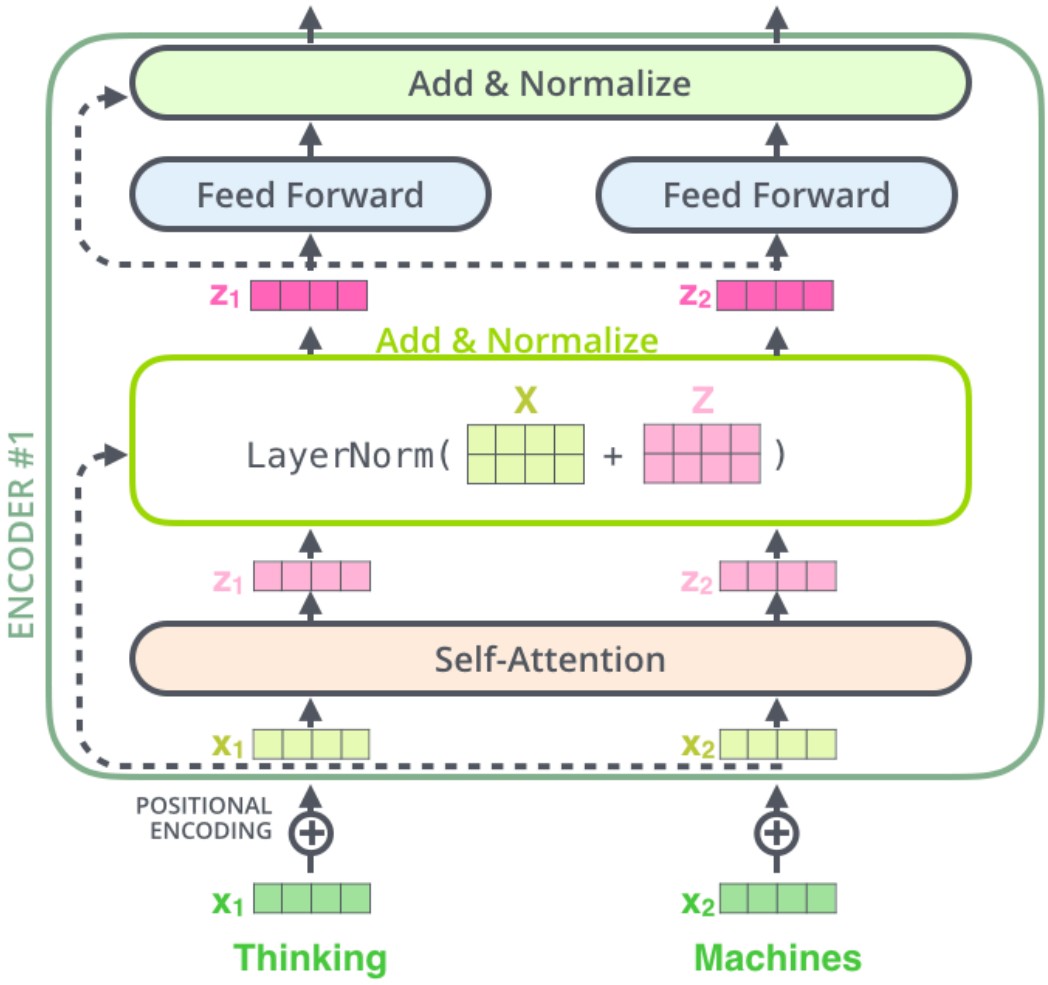
Settings: d = 50

The value of each positional encoding depends on the *position* (*pos*) and *dimension* (*d*). We calculate result for every *index* (*i*) to get the whole vector.

Dodatkowe źródło do nauki: https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers



2 Encoder:



Składa się z:

- **Multi-Head Self-Attention**
- **Add & Norm**
- **Feed Forward Network**
- **Add & Norm**

In [3]:
```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, dim_feedforward, dropout=0.1):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.dropout = nn.Dropout(dropout)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)

    def forward(self, src):
        attn_output, _ = self.self_attn(src, src, src)
        src = self.norm1(src + self.dropout1(attn_output))
        ff_output = self.linear2(self.dropout(F.relu(self.linear1(src))))
        src = self.norm2(src + self.dropout2(ff_output))
        return src
```

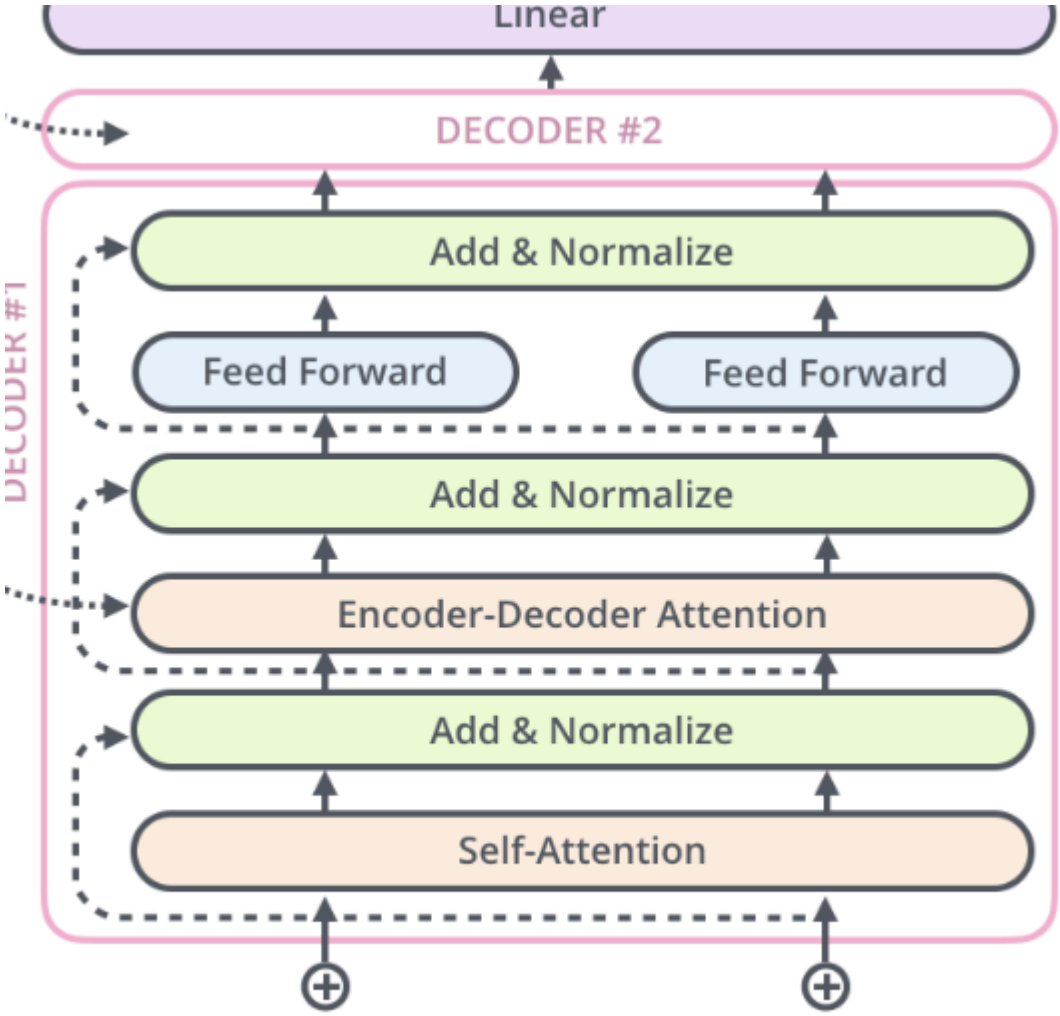
Przykład przepływy danych:

In [4]:
```
d_model = 64
n_heads = 8
dim_feedforward = 256
seq_len = 10
batch_size = 2

encoder_layer = TransformerEncoderLayer(d_model, n_heads, dim_feedforward
x = torch.rand(seq_len, batch_size, d_model)
out = encoder_layer(x)
print("Wejście:", x.shape)
print("Wyjście:", out.shape)
```

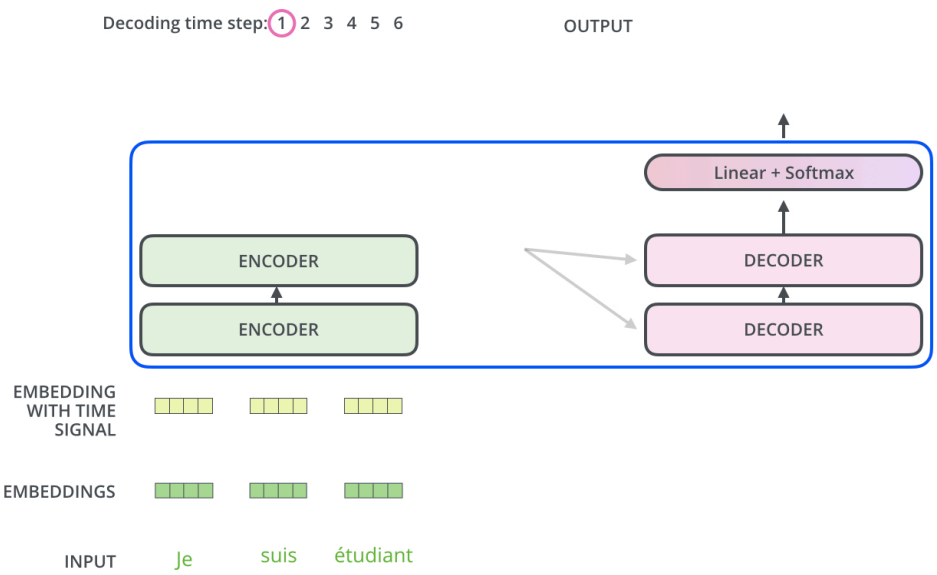
```
Wejście: torch.Size([10, 2, 64])
Wyjście: torch.Size([10, 2, 64])
```

3 - Decoder



Decoder posiada dwa mechanizmy attention:

- **Masked Self-Attention** (żeby nie podglądać przyszłych tokenów),
- **Encoder-Decoder Attention** – „spojrzenie" na kontekst z encodera.



Przypomnienie:

Encoder przetwarza całe wejście (np. zdanie w języku źródłowym) i tworzy zestaw reprezentacji — w postaci par (K,V).

Decoder generuje wyjście (np. tłumaczenie), token po tokenie, i korzysta z tych reprezentacji, żeby „patrzeć" na wejście.

Teraz w decoderze będziemy tworzyć nasze Q (query!)

Animacja:

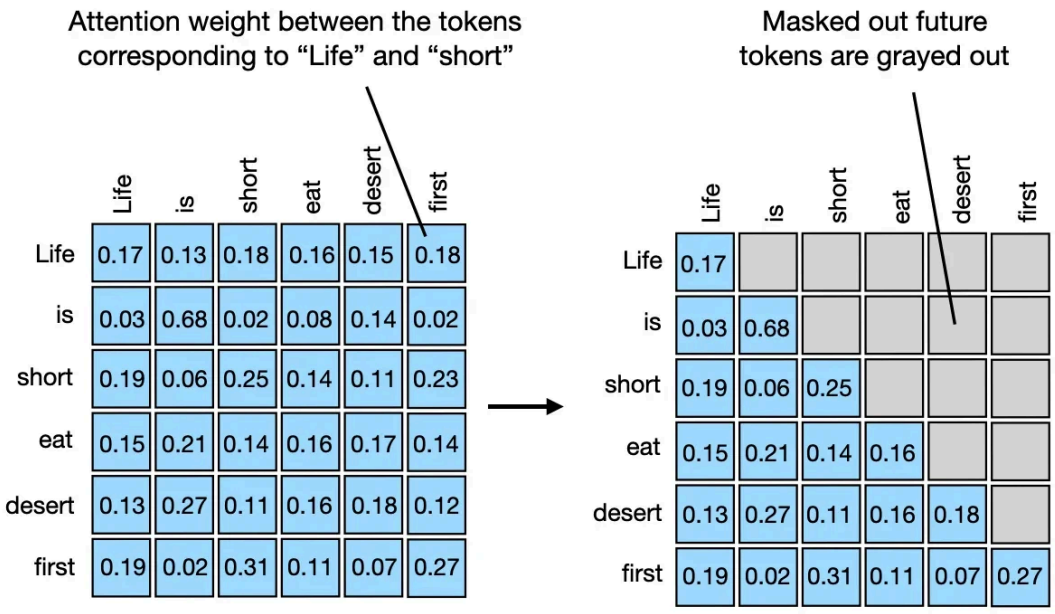
https://jalammar.github.io/images/t/transformer_decoding_1.gif

https://jalammar.github.io/images/t/transformer_decoding_2.gif

Masked Self-Attention:

- Pozwala każdemu tokenowi w sekwencji wyjściowej „zajrzeć" na poprzednie tokeny, ale nie na przyszłe.
- Dzięki temu model może generować tekst autoregsywnie — token po tokenie.

Tak jak w zwykłej rozmowie - wiemy co zostało powiedziane, ale nie wiemy co będzie (możemy tylko zgadywać)



Encoder–Decoder Attention (Cross-Attention)

- Pozwala dekoderowi „patrzeć” na wejście przetworzone przez enkoder.
- W tym miejscu łączy się informacja o tym, co już wygenerowano (z dekodera) i co było w wejściu (z enkodera).

W ten sposób dekoder może „zapytać” wejście o to, które słowa są istotne dla bieżącego etapu generacji.

Robimy attention: K i V z encodera a Q z poprzedniego decodera

In [5]:
```
class TransformerDecoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, dim_feedforward, dropout=0.1):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, n_heads, dropout=
        self.cross_attn = nn.MultiheadAttention(d_model, n_heads, dropout
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.dropout = nn.Dropout(dropout)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)
        self.dropout3 = nn.Dropout(dropout)

    def forward(self, tgt, memory):
        # Masked self-attention
        tgt2, _ = self.self_attn(tgt, tgt, tgt)
        tgt = self.norm1(tgt + self.dropout1(tgt2))

        # Encoder-Decoder attention
        tgt2, _ = self.cross_attn(tgt, memory, memory)
        tgt = self.norm2(tgt + self.dropout2(tgt2))

        # Feed Forward
        tgt2 = self.linear2(self.dropout(F.relu(self.linear1(tgt))))
```

```
        tgt = self.norm3(tgt + self.dropout3(tgt2))
        return tgt
```

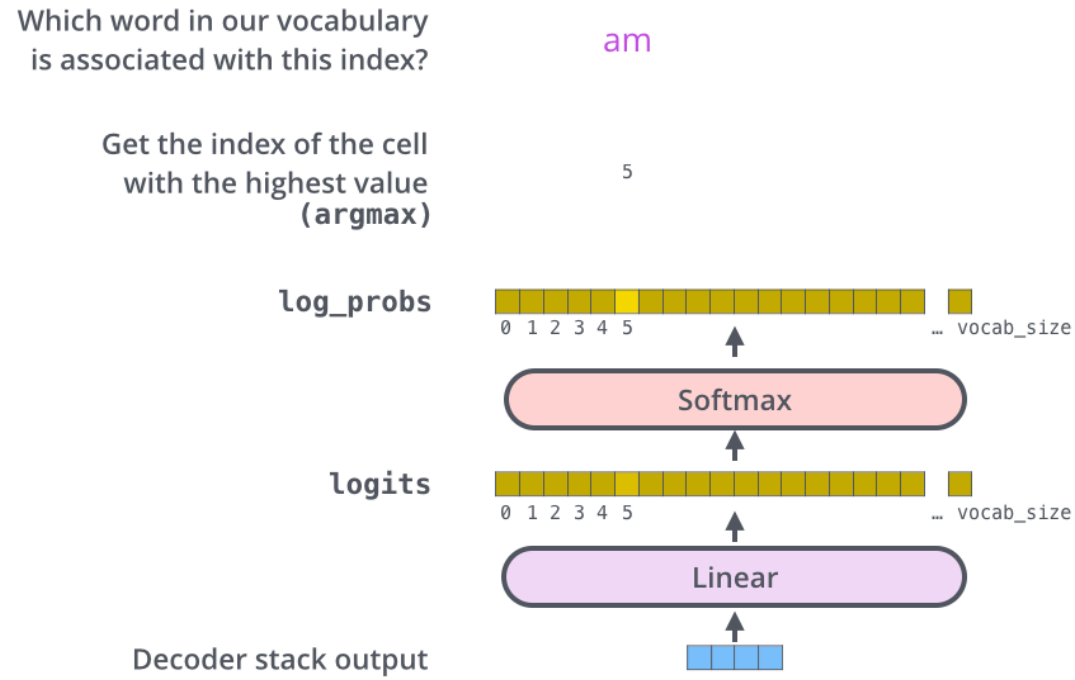
4 - Linear i Softmax - wyjście

Na koniec (po wszystkich encoderach i decoderach) jako ostatnie wyjście z osttaniego decodera mamy jakiś wektor liczb

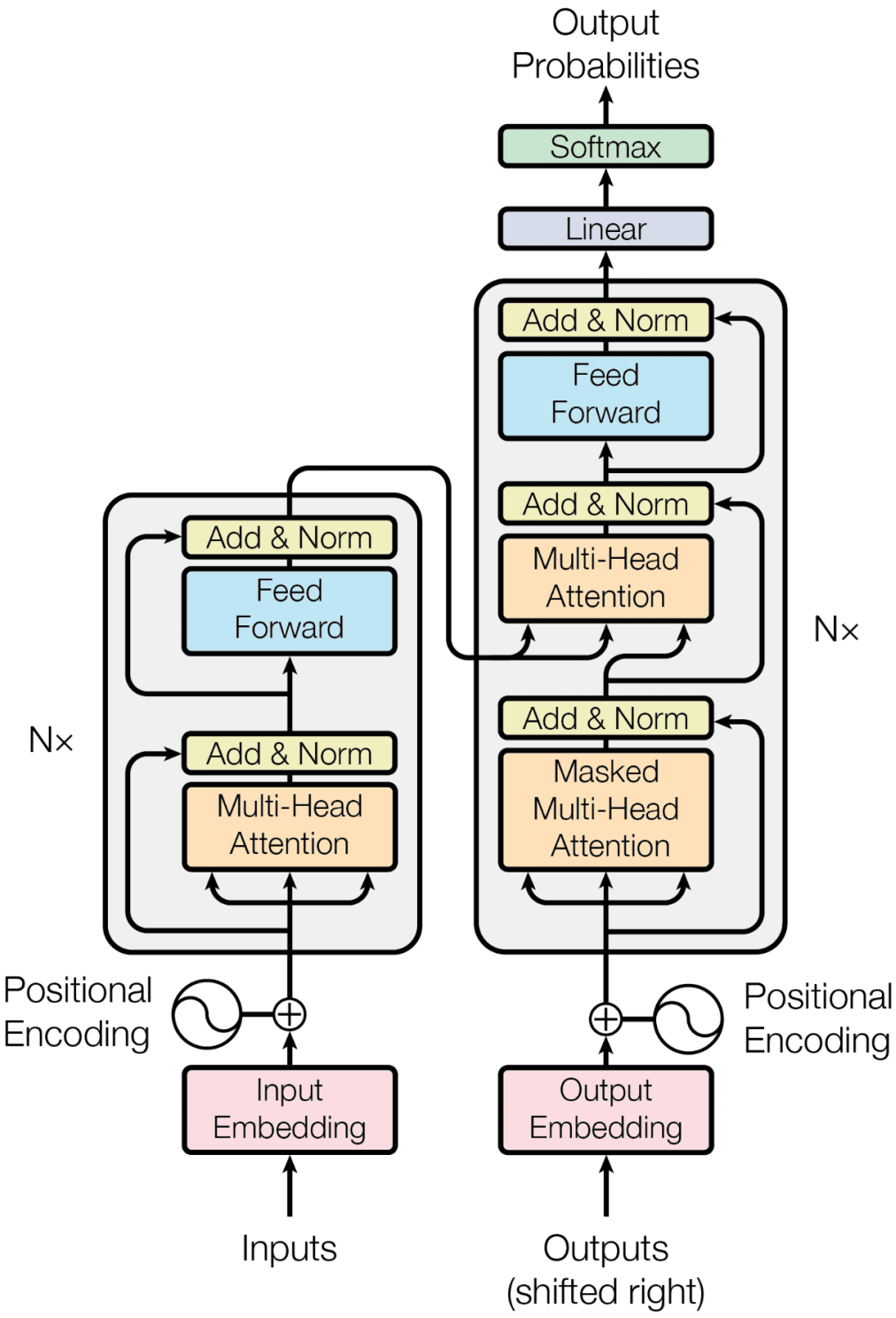
Model następnie przekształca to na przestrzeń słów słownika - surowe logitsy.

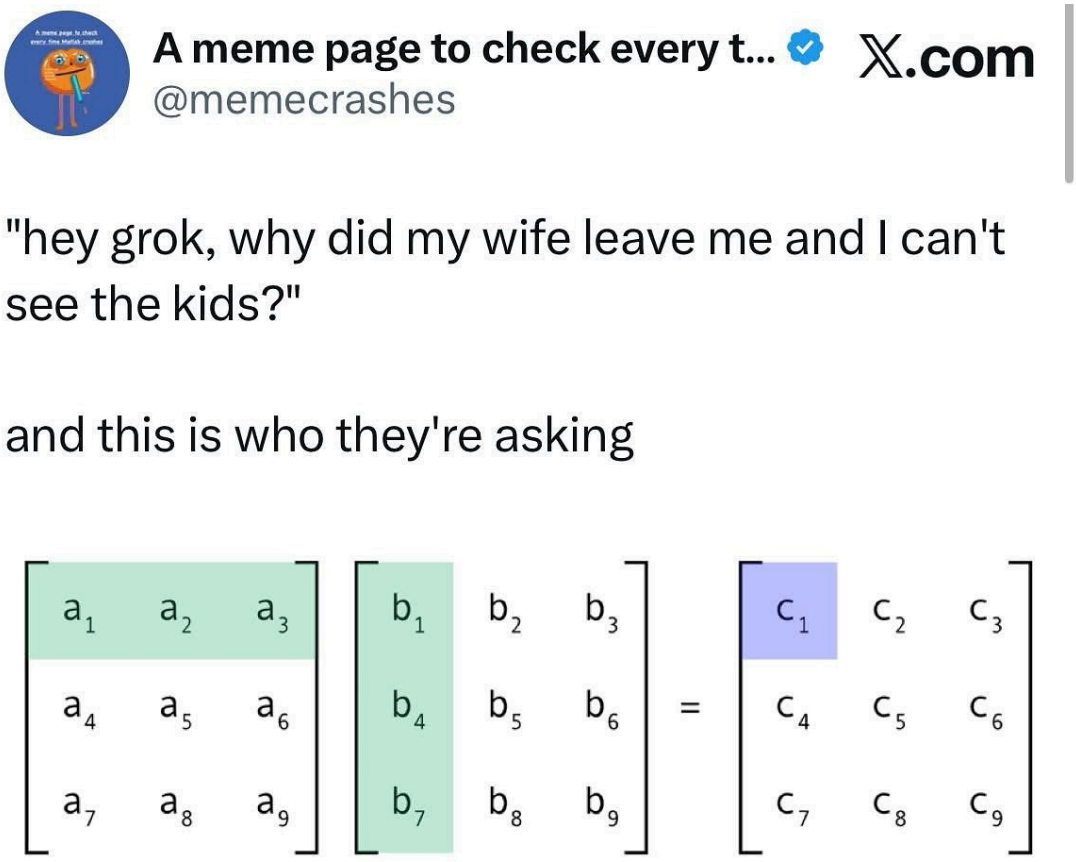
Logity to mogą być losowe liczby - należy więc je przekształcić w prawdopodobieństwa - Softmax

Wybieramy, które słowo jest najbardziej prawdopodobne



I oto cała tajemnica transformerów :)


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)



Zadania

Zadanie 1

Dodaj Masking do Self-Attention (dla dekodera)

- Napisz funkcję generate_subsequent_mask(size) — tworzącą trójkątną maskę (1 tam, gdzie można patrzeć, 0 gdzie nie).
- Dodaj tę maskę do obliczeń QK^T przed softmaxem (ustawiając zabronione pozycje na -inf).

In [6]:
```
import torch
import torch.nn.functional as F
import math

def generate_subsequent_mask(size):
    return torch.tril(torch.ones(size, size))

def masked_attention(Q, K, V, mask):
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(Q.size(-1))
    scores = scores.masked_fill(mask.to(device=scores.device, dtype=torch
    attn = F.softmax(scores, dim=-1)
    output = torch.matmul(attn, V)
    return output, attn

Q = torch.rand(1, 4, 8)
K = torch.rand(1, 4, 8)
```

```
V = torch.rand(1, 4, 8)

mask = generate_subsequent_mask(4)
print("Mask:\n", mask)

output, attn = masked_attention(Q, K, V, mask)

print("\nAttention weights (softmaxed):\n", attn)
print("\nOutput shape:", output.shape)
```

```
Mask:
 tensor([[1., 0., 0., 0.],
        [1., 1., 0., 0.],
        [1., 1., 1., 0.],
        [1., 1., 1., 1.]])

Attention weights (softmaxed):
 tensor([[[1.0000, 0.0000, 0.0000, 0.0000],
         [0.4362, 0.5638, 0.0000, 0.0000],
         [0.2367, 0.3489, 0.4144, 0.0000],
         [0.1999, 0.2381, 0.3555, 0.2065]]])

Output shape: torch.Size([1, 4, 8])
```

Zadanie 2

Zbuduj i przetrenuj własnego mini-Transformera w PyTorchu

Parametry modelu:

In [7]:
```
import torch
from torch import nn
import torch.nn.functional as F

d_model = 64
num_heads = 4
num_layers = 2
vocab_size = 20
max_len = 10
device = "cuda" if torch.cuda.is_available() else "cpu"
```

Sztuczne dane:

In [8]:
```
def generate_data(batch_size=32, seq_len=6):
    src = torch.randint(1, vocab_size, (batch_size, seq_len))
    tgt = src.clone()
    return src, tgt

src, tgt = generate_data()
print(src[0])
```

```
tensor([10,  7, 13,  2,  6,  7])
```

In [9]:
```
import math
torch.set_num_threads(2)

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len):
```

```
        super().__init__()
        positions = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-math.log(100
        pe = torch.zeros(1, max_len, d_model)
        pe[0, :, 0::2] = torch.sin(positions * div_term)
        pe[0, :, 1::2] = torch.cos(positions * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

class MiniTransformer(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads, num_layers, max_le
        super().__init__()
        self.d_model = d_model
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_len)
        self.transformer = nn.Transformer(
            d_model=d_model,
            nhead=num_heads,
            num_encoder_layers=num_layers,
            num_decoder_layers=num_layers,
            dim_feedforward=4 * d_model,
            dropout=0.0,
            batch_first=True
        )
        self.output_layer = nn.Linear(d_model, vocab_size)

    def forward(self, src, tgt_input):
        src_emb = self.positional_encoding(self.embedding(src) * math.sqr
        tgt_emb = self.positional_encoding(self.embedding(tgt_input) * ma
        tgt_mask = nn.Transformer.generate_square_subsequent_mask(tgt_inp
        hidden = self.transformer(src_emb, tgt_emb, tgt_mask=tgt_mask)
        return self.output_layer(hidden)

model = MiniTransformer(vocab_size, d_model, num_heads, num_layers, max_l
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

num_steps = 1000
model.train()
for step in range(num_steps):
    src, tgt = generate_data()
    src, tgt = src.to(device), tgt.to(device)
    tgt_input = torch.cat([torch.zeros_like(tgt[:, :1]), tgt[:, :-1]], di
    optimizer.zero_grad()
    logits = model(src, tgt_input)
    loss = criterion(logits.reshape(-1, vocab_size), tgt.reshape(-1))
    loss.backward()
    optimizer.step()
    if (step + 1) % 200 == 0:
        accuracy = (logits.argmax(dim=-1) == tgt).float().mean().item()
        print(f"Step {step + 1}/{num_steps} | loss: {loss.item():.4f} | a

model.eval()
src, tgt = generate_data(batch_size=1)
src = src.to(device)
with torch.no_grad():
    generated = torch.zeros((1, 1), dtype=torch.long, device=device)
    for _ in range(src.size(1)):
```

```
        logits = model(src, generated)
        next_token = logits[:, -1].argmax(dim=-1, keepdim=True)
        generated = torch.cat([generated, next_token], dim=1)

print("Wejście:", src[0].tolist())
print("Oczekiwane:", tgt[0].tolist())
print("Wygenerowane:", generated[0, 1:].tolist())
```

```
Step 200/1000 | loss: 1.4792 | accuracy: 41.15%
Step 400/1000 | loss: 0.7702 | accuracy: 68.75%
Step 600/1000 | loss: 0.3112 | accuracy: 88.02%
Step 800/1000 | loss: 0.1758 | accuracy: 92.19%
Step 1000/1000 | loss: 0.1432 | accuracy: 95.31%
Wejście: [8, 11, 4, 13, 1, 3]
Oczekiwane: [8, 11, 4, 13, 1, 3]
Wygenerowane: [8, 11, 4, 13, 1, 3]
```